



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): ING. ALDO HERNANDEZ FONTES

Asignatura: Modelos de programación orientada a objetos

Grupo: 7

No de Práctica(s): 5

Integrante(s): Rangel Perez Diego Ivan

Figueroa Regules Irvin Ivan

Torres Sanchez Efren

No. de lista o brigada:

Semestre: 2027-1

Fecha de entrega: 13/09/2026

Observaciones:

CALIFICACIÓN: _____

OBJETIVOS:

Implementar programas utilizando estructuras de selección en un lenguaje orientado a objetos.

Descripción del problema:

El sistema recibe los datos de una compra y devuelve cuánto debe pagar el cliente. Los requisitos son existen 3 tipos de membresía:

PLATA: otorga el 5% de descuento

ORO: otorga el 8% de descuento

DIAMANTE: otorga el 12% de descuento

Hay promociones por día, los días Lunes y Jueves, las membresías tienen un descuento del 12%; Martes, las membresías Oro y Diamante tienen un 10% de descuento sobre lo ya rebajado. Además existe la promoción de cumpleaños la cual hace un 50% de descuento del monto total (al hacer este descuento, los descuentos de membresía y descuentos de día no se aplican).

Diseño de la solución:



Implementación del proyecto:

Fragmentos relevantes del código:

MAIN

```
package practica5;

import java.util.Scanner;

/**
 *
 * @author irvin
 */
public class Main {
    public static void main(String[] args)
    {
        try (Scanner sc = new Scanner(System.in)) {
            int option;
```

- Prepara el entorno para ejecutar la aplicación de consola.

- Inicializa el lector de teclado (sc) dentro de un try automático, garantizando que el sistema liberará los recursos de entrada al salir del programa sin necesidad de llamar a sc.close().

```
do {  
    System.out.println("1. Calcular compra");  
    System.out.println("2. Salir");  
    opcion = sc.nextInt();  
  
    if (opcion == 1)  
    {  
        System.out.println("Monto de la compra:");  
        double monto = sc.nextDouble();  
    }  
}
```

Muestra el menú interactivo en consola y se queda esperando a que el usuario teclee una opción. Si la opción leída es 1, solicita al usuario el importe total de la compra y guarda ese valor numérico decimal en la variable monto.

```
System.out.println("Membresia (1=Plata, 2=Oro, 3=Diamante):");  
int numMembresia = sc.nextInt();  
  
Membresia membresia;  
switch (numMembresia) {  
    case 1 -> membresia = Membresia.PLATA;  
    case 2 -> membresia = Membresia.ORO;  
    case 3 -> membresia = Membresia.DIAMANTE;  
    default -> {  
        System.out.println("Membresia invalida");  
        continue;  
    }  
}
```

Le pide al usuario ingresar un número entre 1 y 3 para la membresía.

El switch traduce esa entrada numérica a la constante fija correspondiente del enum (PLATA, ORO o DIAMANTE). Si el usuario escribe una opción incorrecta (por ejemplo, 8), cae en default, imprime un aviso de error y la palabra continue reinicia de inmediato el ciclo del menú sin pedir los datos restantes.

```

    System.out.println("¿Es su cumpleaños? (s/n):");
    boolean esCumpleanos = sc.next().equalsIgnoreCase("s");

    double precio = new Compra(monto, membresia, dia, esCumpleanos).calcularPrecio();

    if (precio == -1)
    {
        System.out.println("Monto invalido");
    } else
    {
        System.out.println("Total a pagar: $" + precio);
    }
    }
    } while (opcion != 2);
}
}
}

```

Lee el texto ingresado por el usuario ("s" o "n") y lo convierte en un valor booleano (true o false). Instancia la clase Compra pasándole los 4 datos capturados, e invoca de inmediato su método .calcularPrecio(). Si el precio devuelto es -1, muestra "Monto invalido". En caso contrario, despliega el precio final ya calculado y redondeado. Mantiene la aplicación en ejecución repitiendo el menú hasta que la variable opcion sea igual a 2.

COMPRA

```

package practicas;

public class Compra {
    public double monto;
    public Membresia membresia;
    public Dia dia;
    boolean esCumpleanos;
}

```

Declaración de la clase Compra y sus cuatro atributos principales: monto (el importe inicial en formato numérico decimal), membresia (objeto de tipo Membresia), dia (objeto de tipo Dia) y esCumpleanos (un valor booleano true/false). Estos atributos guardan la información de la transacción a procesar.

```
public Compra(double monto, Membresia membresia, Dia dia, boolean esCumpleaños) {  
    this.monto = monto;  
    this.membresia = membresia;  
    this.dia = dia;  
    this.esCumpleaños = esCumpleaños;  
}
```

Constructor de la clase Compra. Recibe como parámetros el monto, la membresía, el día y la condición de cumpleaños, asignándolos directamente a las variables internas de la instancia usando la palabra reservada this.

```
public double calcularPrecio()  
{  
    if (monto <= 0) {  
        return -1;  
    }  
  
    double precio = monto;
```

Define el método calcularPrecio(), llama a una condición de control if (monto <= 0) e inicializa la variable local precio.

- Revisa si el monto ingresado es menor o igual a cero. Si lo es, corta la ejecución inmediatamente regresando **-1** como señal/código de error.
- Si el monto es válido (mayor a 0), crea la variable **precio** y le asigna el valor base de **monto** para empezar a aplicarle los descuentos.

```

if (esCumpleaños)
{
    // Cumpleaños: 50% y no se aplican los demás descuentos
    precio = precio * 0.50;
}
else
{
    // Descuento por membresía (Plata: 5%, Oro: 8%, Diamante: 12%)
    switch (membresia)
    {
        case PLATA -> precio = precio * 0.95;
        case ORO -> precio = precio * 0.92;
        case DIAMANTE -> precio = precio * 0.88;
    }

    // Descuento por día de promoción (sobre lo ya rebajado)
    switch (dia)
    {
        case MARTES -> {
            // Martes: Oro y Diamante tienen 10% adicional
            if (membresia == Membresia.ORO || membresia == Membresia.DIAMANTE) {
                precio = precio * 0.90;
            }
        }
        case LUNES, JUEVES -> // Lunes y Jueves: 12% adicional para todas las membresías
            precio = precio * 0.88;
        default -> {
        }
    }
}
}

```

Llama a la estructura condicional if-else, a dos estructuras de selección switch que evalúan constantes de los enum, y a multiplicaciones de punto flotante para ajustar precio.

- **Rama if (Cumpleaños):** Si esCumpleaños es true, aplica directo el 50% de descuento ($\text{precio} * 0.50$) e ignora por completo el bloque else.
- **Rama else (Días y Membresías):** Si no es cumpleaños, primero evalúa el switch (membresia) y multiplica por 0.95, 0.92 o 0.88 según la categoría del cliente.
- Inmediatamente después, ejecuta un segundo switch (dia) que calcula un descuento extra **acumulado** sobre la cantidad resultante del primer descuento:
 - Si es MARTES y la membresía es ORO o DIAMANTE, descuenta un 10% adicional ($* 0.90$).
 - Si es LUNES o JUEVES, aplica un 12% adicional ($* 0.88$) a cualquier cliente.

```

    return Math.round(precio * 100) / 100.0;
}
}

```

Llama a la función estática `Math.round()` de la biblioteca estándar de Java.

Toma el precio final que puede tener varios decimales, lo multiplica por 100, lo redondea al entero más cercano y lo divide entre 100.0. Esto limpia los decimales sobrantes y devuelve el precio exacto en formato de dos decimales (pesos y centavos).

MEMBRESIA

```
package practica5;

public enum Membresia {
    PLATA,
    ORO,
    DIAMANTE,
}
```

Declara el enum `Membresia`. Establece de forma rígida los tres únicos niveles de membresía permitidos por el sistema (PLATA, ORO y DIAMANTE).

DIA

```
package practica5;

public enum Dia {
    LUNES,
    MARTES,
    MIERCOLES,
    JUEVES,
    VIERNES,
    SABADO,
    DOMINGO,
}
```

Declara un tipo enumerado (enum) llamado `Dia`. Su función es definir un conjunto fijo y restringido de constantes que representan los días de la semana. Esto evita el uso de cadenas de texto arbitrarias o números mágicos, asegurando que solo se puedan procesar días válidos en la aplicación.

Funcionamiento general del programa:

Pruebas y resultados:

Membresía Plata, con descuento del día Lunes y Jueves.

```
1. Calcular compra
2. Salir
1
Monto de la compra:
100
Membresia (1=Plata, 2=Oro, 3=Diamante):
1
Dia (1=Lunes, 2=Martes, 3=Miercoles, 4=Jueves, 5=Viernes, 6=Sabado, 7=Domingo):
1
Es su cumpleaños? (s/n):
n
Total a pagar: $83.6
1. Calcular compra
2. Salir
1
Monto de la compra:
100
Membresia (1=Plata, 2=Oro, 3=Diamante):
1
Dia (1=Lunes, 2=Martes, 3=Miercoles, 4=Jueves, 5=Viernes, 6=Sabado, 7=Domingo):
4
Es su cumpleaños? (s/n):
n
Total a pagar: $83.6
```

Membresía Oro y Diamante con descuento de día martes.

```
1. Calcular compra
2. Salir
1
Monto de la compra:
100
Membresia (1=Plata, 2=Oro, 3=Diamante):
2
Dia (1=Lunes, 2=Martes, 3=Miercoles, 4=Jueves, 5=Viernes, 6=Sabado, 7=Domingo):
2
Es su cumpleaños? (s/n):
n
Total a pagar: $82.8
1. Calcular compra
2. Salir
1
Monto de la compra:
100
Membresia (1=Plata, 2=Oro, 3=Diamante):
3
Dia (1=Lunes, 2=Martes, 3=Miercoles, 4=Jueves, 5=Viernes, 6=Sabado, 7=Domingo):
2
Es su cumpleaños? (s/n):
n
Total a pagar: $79.2
```

Descuento de cumpleaños, se juntan varios descuentos pero al agregar el de cumpleaños los otros se descartan.

```
1. Calcular compra
2. Salir
1000
1. Calcular compra
2. Salir
1
Monto de la compra:
1000
Membresia (1=Plata, 2=Oro, 3=Diamante):
3
Dia (1=Lunes, 2=Martes, 3=Miercoles, 4=Jueves, 5=Viernes, 6=Sabado, 7=Domingo):
1
Es su cumpleaños? (s/n):
s
Total a pagar: $500.0
```

Conclusiones:

En esta práctica se cumplieron satisfactoriamente todos los objetivos planteados al desarrollar un algoritmo capaz de procesar reglas de negocio complejas y acumulativas. Se validó correctamente la aplicación de descuentos en cascada para días de promoción y membresías, así como la jerarquía de exclusividad en la regla de cumpleaños. El uso de bucles de control (do-while) y validaciones de entrada (monto ≤ 0) permitió crear un sistema interactivo capaz de gestionar errores de usuario de forma controlada.

Referencias:

Deitel, P., & Deitel, H. (2017). Java: How to program (11th ed.). Pearson.

Hernández Fontes, A. (2026). *Modelos de programación orientada a objetos. Unidad I – Tema 5: Tipos de clases* [Diapositivas en PDF]. Google Classroom.

Hernández Fontes, A. (2026). *Modelos de programación orientada a objetos. Unidad I – Temas 6 y 7: Estructuras de control* [Diapositivas en PDF]. Google Classroom.

Google. (2026). *Gemini 2.5* [Modelo de lenguaje de gran tamaño].
<https://gemini.google.com>